

Machine learning in finance : Theoretical foundations

Huyên PHAM

Ecole Polytechnique, CMAP

<https://sites.google.com/site/phamxuanhuyen/>

Email : huyen.pham@polytechnique.edu or pham@ensae.fr

Master M2MO, SFA
Mastère spécialisé FGR, DS
ENSAE 3A
Year 2025-2026

Lecture 3 : Introduction to (deep) Reinforcement learning and financial applications

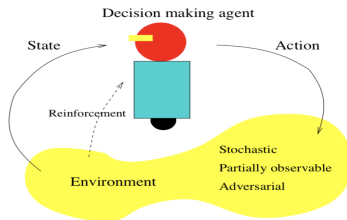
- 1 Foundations of RL
- 2 Some financial applications

From Model-Based to Reinforcement Learning in Finance

- Model-based methods rely on an assumed transition model $P(x, a, dx')$.
- Model misspecification and parameter uncertainty $\Rightarrow$ model risk.
- RL learns optimal strategies directly from interaction with the environment.

Reinforcement Learning :

branch of ML dealing with the learning of optimal strategies in a model-free setting by interacting with the unknown environment by *trial and error* :



Model-Based vs Model-Free Control

Lecture 2 :

- Known transition model $P(x, a, dx')$
- Dynamic programming
- Deterministic policies

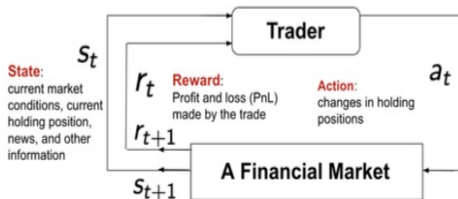
Lecture 3 :

- Unknown transition model
- Learning from interaction
- Randomized policies

Classical applications

- Backgammon
- Robotics
- Production manufacturing optimization
- •
- An AI success story : Video Games, AlphaZero

RL in finance



RL in the financial context :

- A trader (investor) operates in an (**unknown**) environment
- She takes actions $(a_t)_t$ which influences future state (market impact) $\rightarrow s_{t+1}$
- Reward is measured by a signal (r_t) , e.g. PNL, return.
- **Goal** : select actions to maximize expected rewards.

Example : optimal execution

Problem description :

- Optimal execution of large (buy or sell) positions over a given trading period.
- Trade-off between : (1) fast trading speed with less profitable execution prices due to market impact, (2) slow trading with risk of exposure to unknown market fluctuations.

RL approach

- **State variable** : mid-price, inventory, and other market conditions
- **Action** : number of shares to execute at any time
- **Reward** : cash inflow, Sharpe ratio, PnL.

Outline

1 Foundations of RL

2 Some financial applications

Classification of RL algorithms

In a model-free setting, the different learning framework based on samples and observations of state/reward are

- **Online learning** : the agent is in state X_t , take an action α_t , observes the reward $f_t = f(x_t, \alpha_t)$, is in the new state $X_{t+1} \sim P_t(X_t, \alpha_t)$, and proceeds forward again.
- **Episodic learning** : Sequence of episodes where on each episode one follows a policy during a certain period for exploration, and one initializes again the state and starts a new episode.
- **Learning with generative model** : Simulator of the environment that takes as input at any time t state/action (x, a) , and generates as output the reward/next state (f_t, x') .

► In this context, there are mainly two classes of RL methods :

- **Value-based methods**

- Learn a representation of the value and Q functions
- Policy is then derived implicitly from the value function

- **Policy-based methods**

- Learn directly a representation of the policy
- Value function is estimated afterwards from the learnt policy.

Evaluation criterion

For RL with finite horizon, accuracy is evaluated in terms of **number of episodes**. Denote $(V_t^{(n)})_t$ as the estimated value function at episode n .

Asymptotic convergence :

$$\sup_{x \in \mathcal{X}} \sum_{t=0}^T |V_t^{(n)}(x) - V_t(x)| \rightarrow 0, \quad \text{as } n \rightarrow \infty.$$

Sample complexity : number of episodes n to reach

$$\sup_{x \in \mathcal{X}} \sum_{t=0}^T |V_t^{(n)}(x) - V_t(x)| \leq \varepsilon$$

in high probability or under expectation.

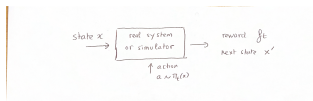
Temporal-difference (TD) learning

For a policy π , recall the Bellman equation satisfied by V^π :

$$V_t^\pi(x) = \mathbb{E}_{\substack{a \sim \pi_t(x) \\ x' \sim P_t(x,a)}} [f(x,a) + V_{t+1}^\pi(x')].$$

► Estimate V^π by **stochastic approximation** (Robbins-Monro algorithm) :

- Initialize at episode $k = 0$ an estimate $\hat{V}^\pi$ of V^π
- Generate (from real system or simulator) episodic trajectories/reward samples (t, x, a, f_t, x') from policy π , $t = 0, \dots, T-1$, f_T final reward



- Update estimate $\hat{V}^\pi$ at the next episode by

$$\hat{V}_t^\pi(x) \leftarrow (1 - \eta) \underbrace{\hat{V}_t^\pi(x)}_{\text{current estimate}} + \eta \left[\underbrace{f_t + \hat{V}_{t+1}^\pi(x')}_{\text{prediction at next state}} \right] \quad (1)$$

with $\hat{V}_{t+1}^\pi(x') = f_T$ when $t = T-1$, and where $\eta = \eta_t(x, a) \in (0, 1)$ is the learning rate. Updating rule (1) is also written as

$$\hat{V}_t^\pi(x) \leftarrow \hat{V}_t^\pi(x) + \underbrace{\eta [f_t + \hat{V}_{t+1}^\pi(x') - \hat{V}_t^\pi(x)]}_{\text{TD error } \delta_t},$$

and usually referred to as **TD(0) algorithm**.

Variations of TD

- TD(1) (Monte-Carlo method) : update value function estimate according to

$$\hat{V}_t^\pi(x) \leftarrow \hat{V}_t^\pi(x) + \eta \sum_{s=t}^{T-1} \delta_s$$

- More generally TD(λ) for $\lambda \in [0, 1]$:

$$\hat{V}_t^\pi(x) \leftarrow \hat{V}_t^\pi(x) + \eta \sum_{s=t}^{T-1} \lambda^{s-t} \delta_s$$

Bias-Variance Tradeoff

- $\lambda \approx 0$: low variance, high bias
- $\lambda \approx 1$: low bias, high variance

Q-learning

Stochastic approximation of the Bellman equation for the Q -function :

$$Q_t(x, a) = f(x, a) + \mathbb{E}_{x' \sim P_t(x, a)} \left[\sup_{a' \in A} Q_{t+1}(x', a') \right].$$

► **Q-learning algorithm :**

- Initialize at episode $k = 0$ an estimate $\hat{Q}$ of Q
- Generate trajectories/reward samples (t, x, a, f_t, x') , $t = 0, \dots, T - 1$, f_T final reward, with action chosen from some **suitable policy** π
- Update estimate $\hat{Q}$ at the next episode by

$$\begin{aligned} \hat{Q}_t(x, a) &\longleftarrow (1 - \eta) \hat{Q}_t(x, a) + \eta \left[f_t + \sup_{a' \in A} \hat{Q}_{t+1}(x', a') \right] \\ &= \hat{Q}_t(x, a) + \underbrace{\eta \left[f_t + \sup_{a' \in A} \hat{Q}_{t+1}(x', a') - \hat{Q}_t(x, a) \right]}_{\text{TD error } \delta_t} \end{aligned}$$

with $\hat{Q}_{t+1}(x', a') = f_T$ when $t = T - 1$.

Properties of Q-learning

Under the Robbins-Monro conditions on the learning rate, and if all states are visited **infinitely often**, then for all $(t, x, a) \in \{0, \dots, T\} \times \mathcal{X} \times A$,

$$\hat{Q}_t(x, a) \rightarrow Q_t(x, a), \quad a.s.$$

Remark.

- All state-action pairs must be visited infinitely often : requires a **suitable exploration policy**.
- Approximate optimal policy is computed **off-line** from

$$\hat{\pi}_t(x) \in \arg \max_{a \in A} \hat{Q}_t(x, a).$$

SARSA : online policy learning

In contrast to Q-learning, SARSA learns policy π^k **on-line** by iterations from

- **Policy evaluation (PE)** : current estimate by TD stochastic approximation of Q^{π^k} satisfying the Bellman equation :

$$Q_t^{\pi^k}(x, a) = f(x, a) + \mathbb{E}_{\substack{x' \sim P_t(x, a) \\ a' \sim \pi_{t+1}^k(x')}} \left[Q_{t+1}^{\pi^k}(x', a') \right]$$

- **Policy improvement/iteration (PI)** : compute e.g. the greedy policy

$$\pi_t^{k+1}(x) \in \arg \max_{a \in A} Q_t^{\pi^k}(x, a), \quad t = 0, \dots, T-1, \quad x \in \mathcal{X}.$$

$$\rightarrow V_t^{\pi^k} \leq V_t^{\pi^{k+1}} \text{ for all } t.$$

See Proof in Annex : [Annex : Proof of PI](#)

Q-learning and SARSA Algorithms for learning optimal policy

For $k = 0, \dots, K$ (number of episodes)

1. Initialize $\hat{Q}_t(x, a)$, $(t, x, a) \in \{0, \dots, T\} \times \mathcal{X} \times A$
2. Set initial state x
3. For $t = 0, \dots, T - 1$ (length of episode)
 - Take action a according to a **suitable exploration policy**
 - Observe reward f_t and next state x'
 - In the case of SARSA, take action a' from PI based on $\hat{Q}$
 - Compute temporal difference :

$$\delta_t = \begin{cases} f_t + Q(x', a') - Q(x, a) & \text{(SARSA)} \\ f_t + \max_{a'} Q(x', a') - Q(x, a) & \text{(Q-learning)} \end{cases}$$

- Update Q-function :

$$\hat{Q}_t(x, a) \leftarrow \hat{Q}_t(x, a) + \eta \delta_t,$$

- $x \leftarrow x'$

Endfor

Suitable policy : Exploitation/exploration trade-off

Two common randomized policies when A is finite

(i) **ε -greedy policy** : take action according to

$$\alpha_t \sim \begin{cases} \arg \max_{a \in A} Q_t(x, a) & \text{with probability } 1 - \varepsilon \\ \mathcal{U}(A), & \text{with probability } \varepsilon \end{cases}$$

$\varepsilon \in (0, 1)$ is a parameter of exploration in the action space A via the uniform distribution $\mathcal{U}(A)$.

(ii) **Soft-max policy** : $\alpha_t \sim \pi_t(x)$ with

$$\pi_t(x, a) = \frac{\exp(Q_t(x, a)/\lambda)}{\sum_{a' \in A} \exp(Q_t(x, a')/\lambda)}, \quad a \in A.$$

The higher $Q_t(x, a)$, the more probability to take action a in state x , and this is weighted by the temperature parameter $\lambda > 0$

- $\lambda \rightarrow \infty$: one chooses each action with same probability
- $\lambda \rightarrow 0$: greedy policy

Connection between Softmax and Shannon entropy

Consider the greedy maximization problem (case when A is finite)

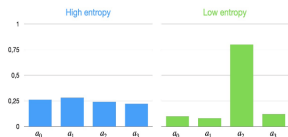
$$\max_{a \in A} Q(a).$$

- **Smooth-max regularization** problem with Shannon entropy :

$$V_\lambda := \max_{\pi \in \mathcal{P}(A)} \mathbb{E}_\pi [Q(\alpha)] + \lambda \mathcal{E}(\pi), \quad (2)$$

where $\lambda > 0$, and the entropy $\mathcal{E}$ is the strictly concave function on $\mathcal{P}(A)$ defined by

$$\mathcal{E}(\pi) := - \sum_{a \in A} \pi(a) \log(\pi(a)) = -\mathbb{E}_\pi [\log \pi(\alpha)]$$



- The solution to (2) is given by the soft-max probability

$$\pi^*(a) = \frac{\exp(Q(a)/\lambda)}{\sum_{a' \in A} \exp(Q(a')/\lambda)}, \quad a \in A,$$

with $V_\lambda = \lambda \log [\sum_{a \in A} \exp(Q(a)/\lambda)]$.

Interpretation

Entropy regularization encourages exploration and smooth policies. Q -learning with entropy regularization is often referred as “soft Q -learning”.

Value-Based Methods : Summary

- TD $\rightarrow$ Policy evaluation
- Q-learning $\rightarrow$ Off-policy optimal control
- SARSA $\rightarrow$ On-policy control

Limitations :

- Large state space $\rightarrow$ function approximation needed
- Continuous action space problematic

Policy-based learning

- In value-based methods, the optimal policy is obtained after the learning of (optimal) value function :
 - Critic algorithm
 - Classical Q -learning works well in practice when state/action space is finite or "small".
- Policy-based learning : model directly policy by a parametric function (e.g. NN) approximator :
 - Stochastic (randomized) policies π_θ of parameter θ with density $a \mapsto \pi_\theta(t, x, a)$ w.r.t. measure ν on action space A .
 - Policy gradient
 - Actor algorithm

Examples of parametrized randomized policies

- Softmax for discrete action space A with density w.r.t. the counting measure

$$\pi_{\theta}(t, x, a) = \frac{\exp(\varphi_{\theta}(t, x, a))}{\sum_{a' \in A} \exp(\varphi_{\theta}(t, x, a'))}, \quad a \in A,$$

where φ_{θ} is a NN. $\rightarrow$ Score function :

$$\nabla_{\theta} \ln \pi_{\theta}(t, x, a) = \nabla_{\theta} \varphi_{\theta}(t, x, a) - \mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \varphi_{\theta}(t, x, .)]$$

- Gaussian distribution for continuous action space A

$$a \mapsto \pi_{\theta}(t, x, a) \quad \text{density of} \quad \mathcal{N}(\varphi_{\theta}(t, x), \sigma^2),$$

where φ_{θ} is a NN. $\rightarrow$ Score function :

$$\nabla_{\theta} \ln \pi_{\theta}(t, x, a) = \frac{(a - \varphi_{\theta}(t, x)) \nabla_{\theta} \varphi(t, x)}{\sigma^2}$$

Policy objective function

- Performance of a policy π_θ :

$$J(\theta) \quad := \quad \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{T-1} f(X_t, \alpha_t) + g(X_T) \right].$$

- Policy based reinforcement learning :
 - Find θ that maximizes $J(\theta)$
 - Gradient ascent method

Gradient representation

Denote by $S = (X_0, \alpha_0, X_1, \dots, \alpha_{T-1}, X_T)$ a trajectory of state/action, and by $R(S)$ the associated total reward :

$$R(S) = \sum_{t=0}^{T-1} f(X_t, \alpha_t) + g(X_T), \quad \text{so that} \quad J(\theta) = \mathbb{E}_{\pi_\theta} [R(S)].$$

Theorem : gradient representation

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[R(S) \sum_{t=0}^{T-1} \nabla_\theta \ln \pi_\theta(t, X_t, \alpha_t) \right].$$

Proof : Log-likelihood trick

Observe that the law $\pi_\theta(ds)$ of $S = (X_0, \alpha_0, \dots, X_T)$ when $X_0 \sim \mu_0$, $\alpha \sim \pi_\theta$, is :

$$\mu_0(dx_0)\pi_\theta(0, x_0, a_0)\nu(da_0)P_0(x_0, a_0, dx_1) \dots \pi_\theta(T-1, x_{T-1}, a_{T-1})\nu(da_{T-1})P_T(x_{T-1}, a_{T-1}, dx_T)$$

hence admits a density :

$$\pi_\theta(ds) = \underbrace{\prod_{t=0}^{T-1} \pi_\theta(t, x_t, a_t) \mu_0(dx_0)}_{p_\theta(s)} \underbrace{\prod_{t=0}^{T-1} \nu(da_t) P_t(x_t, a_t, dx_{t+1})}_{Q(ds)}$$

→

$$J(\theta) = \int R(s) \pi_\theta(ds) = \int R(s) p_\theta(s) Q(ds).$$

►

$$\begin{aligned} \nabla_\theta J(\theta) &= \int R(s) \nabla_\theta p_\theta(s) Q(ds) = \int R(s) [\nabla_\theta \ln p_\theta(s)] p_\theta(s) Q(ds) \\ &= \mathbb{E}_{\pi_\theta} [R(S) \nabla_\theta \ln p_\theta(S)] = \mathbb{E}_{\pi_\theta} \left[R(S) \sum_{t=0}^{T-1} \nabla_\theta \ln \pi_\theta(t, x_t, a_t) \right]. \end{aligned}$$

Reinforce algorithm

For each episode

- Generate a batch of K trajectories $S = (X_0, \alpha_0, \dots, X_T)$, and rewards $(f_0, \dots, f_T)$:
 - Samples α_t^k from $\pi^\theta(t, X_t^k, \cdot)$, X_{t+1}^k from $P_t(X_t^k, \alpha_t^k, \cdot)$, $f_t^k = f(X_t^k, \alpha_t^k)$, $t = 0, \dots, T-1$, $f_T^k = g(X_T^k)$, $\rightarrow R(S^k) = \sum_{t=0}^{T-1} f_t^k + f_T^k$, $k = 1, \dots, K$.
- Update θ according to the policy gradient

$$\theta \leftarrow \theta + \eta \left[\frac{1}{K} \sum_{k=1}^K R(S^k) \sum_{t=0}^{T-1} \nabla_{\theta} \ln \pi_{\theta}(t, X_t^k, \alpha_t^k) \right]$$

Reducing variance with reward baseline

- Policy gradient algorithms require samples over both states and actions of the whole trajectory $\rightarrow$ suffer from high variance
- Observation : subtract a constant baseline b from reward without biasing the gradient

$$\begin{aligned}\mathbb{E}_{\pi_{\theta}}[(R(S) - b)\nabla_{\theta} \ln p_{\theta}(S)] &= \mathbb{E}_{\pi_{\theta}}[R(S)\nabla_{\theta} \ln p_{\theta}(S)] - b\mathbb{E}_{\pi_{\theta}}[\nabla_{\theta} \ln p_{\theta}(S)] \\ &= \nabla_{\theta} J(\theta) - b\nabla_{\theta} \int p_{\theta}(s)Q(ds) = \nabla_{\theta} J(\theta).\end{aligned}\quad (3)$$

- Good choice of baseline would be $\mathbb{E}_{\pi_{\theta}}[R(S)]$, but not easy to compute it ! Anyway, an approximation $\rightarrow$ actor/critic approach

Actor-Critic network approach

- Learns both the value function (the critic) and the policy (the actor) by neural networks.
- Update alternatively the critic and the actor networks
 - Update critic by policy evaluation of the actor
 - Update actor by policy gradient in direction suggested by critic

Actor/Critic gradient representation

Theorem : AC gradient representation

$$\begin{aligned}\nabla_{\theta} J(\theta) &= \sum_{t=0}^{T-1} \mathbb{E}_{\pi_{\theta}} \left[(f(X_t, \alpha_t) + V^{\pi_{\theta}}(t+1, X_{t+1})) \nabla_{\theta} \log \pi_{\theta}(t, X_t, \alpha_t) \right] \\ &= \sum_{t=0}^{T-1} \mathbb{E}_{\pi_{\theta}} \left[(f(X_t, \alpha_t) + V^{\pi_{\theta}}(t+1, X_{t+1}) - V^{\pi_{\theta}}(t, X_t)) \nabla_{\theta} \log \pi_{\theta}(t, X_t, \alpha_t) \right],\end{aligned}\tag{4}$$

where $V^{\pi_{\theta}}$ is the value function associated to the policy π_{θ} .

See Proof in Annex [Annex : Proof of AC representation](#)

Actor-Critic Principle

Actor :

$$\pi_{\theta}$$

Critic :

$$\mathcal{V}_{\phi}$$

TD-error :

$$\delta_t = f_t + \mathcal{V}_{\phi}(t+1, X_{t+1}) - \mathcal{V}_{\phi}(t, X_t)$$

Actor update :

$$\theta \leftarrow \theta + \eta \delta_t \nabla_{\theta} \log \pi_{\theta}$$

Critic update :

$$\phi \leftarrow \phi - \eta \delta_t \nabla_{\phi} \mathcal{V}_{\phi}$$

Interpretation of Actor-Critic

Actor : Improves policy using gradient.

Critic : Approximates value function.

Hybrid of :

- Dynamic programming (Lecture 2)
- Stochastic gradient ascent

Pseudo-code of the episodic offline AC algorithm

Algorithm 1: Actor-Critic (offline)

Input data : Number of episodes E , mini-batch size K , learning rates η^G , η^V for policy and value function estimation; Parametrized family π_θ for randomized policies, and $\mathcal{V}_\phi$ for value function;

Initialization : parameter θ , ϕ ;

for each episode $e = 1, \dots, E$ **do**

 Select a random path $k = 1, \dots, K$;

 Initialize state $X_0^{(k)}$;

for $t = 0, \dots, T - 1$ **do**

 Generate action $\alpha_t^{(k)} \sim \pi_\theta(t, X_t^{(k)}, \cdot)$, and observe state, reward $X_{t+1}^{(k)}, f_t^{(k)}$;

 If $t = T - 1$, set $\mathcal{V}_\phi(t + 1, X_{t+1}^{(k)}) = f_T^{(k)} = g(X_T^{(k)})$, and close the loop;

 Otherwise $t \leftarrow t + 1$;

end

 Compute for path k

$$\Gamma_\theta^{(k)} := \sum_{t=0}^{T-1} (f_t^{(k)} + \mathcal{V}_\phi(t+1, X_{t+1}^{(k)}) - \mathcal{V}_\phi(t, X_t^{(k)})) \nabla_\theta \log \pi_\theta(t, X_t^{(k)}, \alpha_t^{(k)})$$

$$\Delta_\phi^{(k)} := \sum_{t=0}^{T-1} (f_t^{(k)} + \mathcal{V}_\phi(t+1, X_{t+1}^{(k)}) - \mathcal{V}_\phi(t, X_t^{(k)})) \nabla_\phi \mathcal{V}_\phi(t, X_t^{(k)})$$

 Actor update : $\theta \leftarrow \theta + \eta^G \frac{1}{K} \sum_{k=1}^K \Gamma_\theta^{(k)}$; Critic update : $\phi \leftarrow \phi + \eta^V \frac{1}{K} \sum_{k=1}^K \Delta_\phi^{(k)}$;

end

Return : $\pi_\theta, \mathcal{V}_\phi$.

Pseudo-code of the episodic online AC algorithm

Algorithm 2: Actor-Critic (online)

Input data : Number of episodes E , mini-batch size K , learning rates η^G , η^V for policy and value function estimation ; Parametrized family π_θ for randomized policies, and $\mathcal{V}_\phi$ for value function ;

Initialization : parameter θ , ϕ ;

for each episode $e = 1, \dots, E$ **do**

 select a random path $k = 1, \dots, K$;

 Initialize state $X_0^{(k)}$;

for $t = 0, \dots, T - 1$ **do**

 Generate action $\alpha_t^{(k)} \sim \pi_\theta(t, X_t^{(k)}, \cdot)$, and observe state, reward $X_{t+1}^{(k)}, f_t^{(k)}$

 If $t = T - 1$, set $\mathcal{V}_\phi(t + 1, X_{t+1}^{(k)}) = f_T^{(k)} = g(X_T^{(k)})$

 Actor update :

$$\theta \leftarrow \theta + \eta^G (f_t^{(k)} + \mathcal{V}_\phi(t + 1, X_{t+1}^{(k)}) - \mathcal{V}_\phi(t, X_t^{(k)})) \nabla_\theta \log \pi_\theta(t, X_t^{(k)}, \alpha_t^{(k)})$$

 Critic update :

$$\phi \leftarrow \phi + \eta^V (f_t^{(k)} + \mathcal{V}_\phi(t + 1, X_{t+1}^{(k)}) - \mathcal{V}_\phi(t, X_t^{(k)})) \nabla_\phi \mathcal{V}_\phi(t, X_t^{(k)})$$

 If $t = T - 1$, close the loop ; Otherwise $t \leftarrow t + 1$;

end

end

Return : $\pi_\theta, \mathcal{V}_\phi$.

Extension to MDP with exit time

- Case where the horizon is random given by

$$\tau = \inf\{t : X_t \notin \mathcal{O}\} \wedge T,$$

where $\mathcal{O}$ is an open set of $\mathcal{X}$.

- In the policy gradient and actor/critic algorithm, replace T by τ .

Value-based vs policy-based methods

- Main advantage of policy-based methods : **directly optimize for policy**, which is what we care about the most
- Policy-based methods tend to be **more stable** and less prone to failure
- Value-based methods may **suffer from instability** when combined with function approximation.
- Q-learning methods, when they work, tend to be **more sample efficient** than policy-based methods because they can **exploit data from experts or other sources**.

Outline

1 Foundations of RL

2 Some financial applications

Why Reinforcement Learning in Finance?

Useful when :

- Model uncertainty is large
- Market impact difficult to estimate
- Data abundant
- Complex constraints

Applications :

- Optimal execution
- Share repurchase
- Portfolio optimization
- Market making

RL for Financial Control Problems

General structure

State :

X_t = inventory, price, features

Action :

α_t = trade decision

Reward :

$r(X_t, \alpha_t)$

RL learns optimal strategy directly from simulated or historical data.

Optimal stock execution

A broker has to sell N blocks of n shares over finite horizon T .

- Inventory Y_t at time t , starting from $Y_0 = N$ (target $Y_T = 0$).
 - Actions α_t : number of block of shares sold a time t , valued in $A = \{0, 1, 2, 3\}$.
 - $\rightarrow Y_{t+1} = (Y_t - \alpha_t)_+$.
- Stock price dynamics subject to market impact from trades :

$$S_{t+1} = S_t e^{1-\nu\alpha_t} + \sigma S_t \varepsilon_{t+1}, \quad \text{discretized to } M \text{ values}$$

$\rightarrow$ State variable $X = (Y, S)$ valued in $\mathcal{X} = \{0, 1, \dots, N\} \times \{1, \dots, M\}$.

- Reward at time t : Immediate execution revenue minus inventory risk penalty

$$r(X_t, \alpha_t) = n\alpha_t S_t - \lambda n |S_t Y_t|^2.$$

- Q-learning to learn optimal stock execution

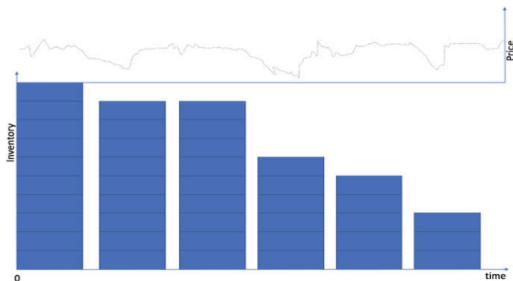


Fig. 9.5 The optimal execution problem: how to break up large market orders into smaller orders with lower market impact? In the finite MDP formulation, the state space is the inventory, shown by the number of blocks, stock price, and time. In this illustration, the agent decides whether to sell $\{0, 1, 2, 3\}$ blocks at each time step. The problem is whether to retain inventory, thus increasing market risk but reduces the market impact, or quickly sell inventory to reduce exposure but increase the market impact

Q-learning table and optimal trading

$$N = 9, M = 12$$

$t = 2$	Price level											
Inventory	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	0	0
3	0	2	0	0	0	0	0	0	0	0	0	0
4	0	2	0	0	0	0	0	0	0	0	0	0
5	0	2	0	0	0	0	0	0	0	0	0	0
6	0	1	0	1	0	0	0	0	0	0	0	0
7	0	3	2	2	0	0	0	0	0	0	0	0
8	0	0	0	2	3	0	3	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	0	0

Stock Repurchase Programs (SRP)

Pricing of buyback options in SRP : contract between a firm and a bank that allows the former to buy back its own shares within a finite period :

- Raise the debt-to-equity ratio
- Improve the earnings per share by reducing the number of outstanding shares
- Alternative way to distribute dividends to shareholders

Problem formulation

A company mandates a bank/trader to buy a quantity B of shares of its own stock S within $[0, T]$.

- Inventory of the trader :

$$dQ_t = \alpha_t dt,$$

where trading rate α valued in $[0, \bar{a}]$, and termination time of the contract :

$$\tau = \inf\{t \geq 0 : Q_t \geq B\} \wedge T.$$

- PnL of the bank : $\text{PnL}_\tau^\alpha = B(Y_\tau - S_\tau) - \lambda(B - Q_\tau)_+$, with
 - TWAP : $Y_t = \frac{1}{t} \int_0^t S_u du$
 - $\lambda > 0$ penalizes whenever the bank fails in her mandate, i.e. when $\tau = T$, $Q_T < B$.

- Price of the SRP contract :

$$P_{SRP} = \sup_{\alpha} \mathbb{E}[\text{PnL}_\tau^\alpha].$$

Remark : one can consider market impact on $S = S^\alpha$, and also transaction costs.

Model implementation

- Price process S with linear permanent price impact

$$dS_t = S_t(\gamma\alpha_t dt + \sigma dW_t),$$

with $\gamma \geq 0$ market impact parameter.

- ▶ Model in dimension 3 with Markov state variables $X = (S, Y, Q)$, and $\mathcal{O} = (0, \infty) \times (0, \infty) \times [0, B)$. Notice that the optimal feedback control is bang-bang between 0 and $\bar{a}$.
- ▶ Actor and critic trained with **softmax randomized policies** with $A = \{0, \bar{a}\}$.
- Architecture of neural networks (NN) : two hidden layers with 8 neurons, linear output and Relu activation functions.

Benchmark

Compare RL results with HJB splitting scheme.

Numerical results on the price (critic)

The PG algorithms take about two minutes to converge, comparable to HJB solver

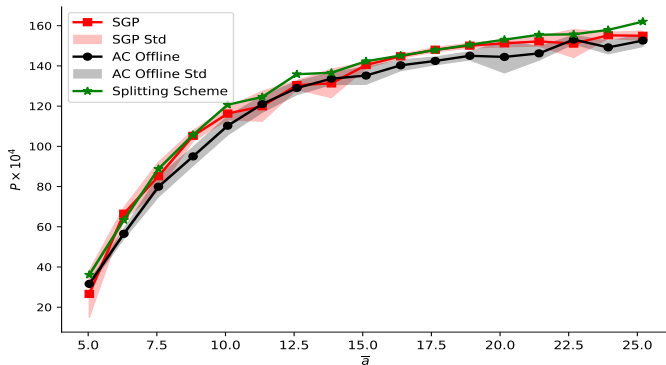


Figure – $P_{SBR} \times 10^4$ in absence of market impact for different values of $\bar{a}$ computed with stochastic gradient policy and actor critic compared to splitting scheme (HJB solver).

Numerical results on the optimal trading rate (actor)

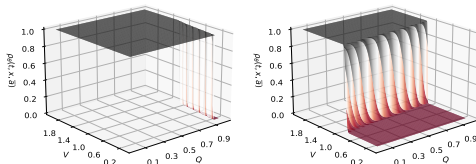


Figure – Optimal policy π_θ in absence of market impact for $S = 1$, $\sigma = 0.2$, $t \simeq T$ (left) and $t = \frac{T}{2}$ (right).

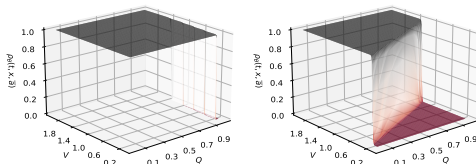


Figure – Optimal policy π_θ with market impact ($\gamma = 0.1$) for $S = 1$, $\sigma = 0.2$, $t \simeq T$ (left) and $t = \frac{T}{2}$ (right).

Evolution of the inventory

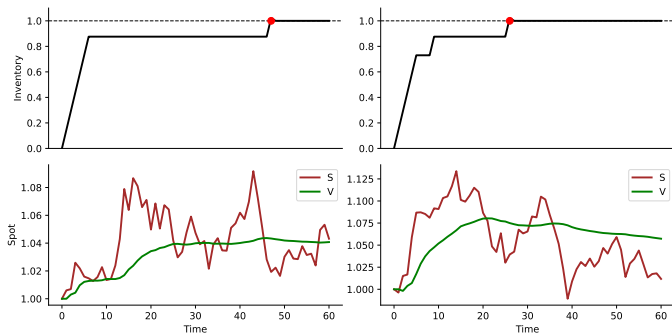


Figure – Optimal repurchase strategy evolution for two price realizations ($\sigma = 0.2$) with market impact ($\gamma = 0.1$)

Comments

Takeaway

- RL matches HJB solver in low dimension.
 - Scales better to high dimension and constraints.
 - Robust to model misspecification.
-
- Many more applications in finance :
 - Portfolio optimization, option pricing and hedging
 - Optimal execution, Market making, Algo trading
 - Robo advising

See list of projects.

Lecture 3 in Perspective

Model-Based (L2) :

- Bellman equation
- Backward recursion
- Deterministic policies

Model-Free (L3) :

- TD / Q-learning
- Policy gradient
- Actor-Critic

Both aim to solve high-dimensional stochastic control in finance.

Concluding remarks

- **Domain knowledge** is important for applying RL method in finance
 - Better understanding of the environment
 - Design problem-dependent state space and action space
 - Develop more sample-efficient algorithms
- **RL method** :
 - RL requires large sample sizes.
 - Exploration can be costly in financial markets.
 - Market-wide learning interactions remain poorly understood.

Indeed, this is true for $t = T$. By backward induction, assume this holds true for $t + 1$. Then, from the Bellman equation for V^{π^k} , $V^{\pi^{k+1}}$:

$$\begin{aligned} V_t^{\pi^k}(x) &\leq \sup_{a \in A} Q_t^{\pi^k}(x, a) = Q_t^{\pi^k}(x, \pi_t^{k+1}(x)) \\ &= f(x, \pi_t^{k+1}(x)) + \mathbb{E}_{x' \sim P_t(x, \pi_t^{k+1}(x))} [V_{t+1}^{\pi^k}(x')] \\ &\leq f(x, \pi_t^{k+1}(x)) + \mathbb{E}_{x' \sim P_t(x, \pi_t^{k+1}(x))} [V_{t+1}^{\pi^{k+1}}(x')] = V_t^{\pi^{k+1}}(x). \end{aligned}$$

Annex : Proof of AC representation

[back](#)

The DP relation for the value function is written as

$$\begin{aligned} V^{\pi_\theta}(t, x) &= \mathbb{E}_{\pi_\theta} [f(X_t, \alpha_t) + V^{\pi_\theta}(t+1, X_{t+1}) | X_t = x] \\ &= \int_{\mathcal{X}} \int_A [f(x, a) + V^{\pi_\theta}(t+1, x')] \pi_\theta(t, x, a) \nu(da) P_t(x, a, dx'). \end{aligned}$$

By differentiating w.r.t. θ and using again the log-likelihood trick, we get

$$\begin{aligned} \nabla_\theta V^{\pi_\theta}(t, x) &= \int_{\mathcal{X}} \int_A \nabla_\theta [V^{\pi_\theta}(t+1, x')] \pi_\theta(t, x, a) \nu(da) P_t(x, a, dx') \\ &\quad + \int_{\mathcal{X}} \int_A [f(x, a) + V^{\pi_\theta}(t+1, x')] [\nabla_\theta \log \pi_\theta](t, x, a) \pi_\theta(t, x, a) \nu(da) P_t(x, a, dx') \\ &= \mathbb{E}_{\pi_\theta} [\nabla_\theta V^{\pi_\theta}(t+1, X_{t+1}) | X_t = x] \\ &\quad + \mathbb{E}_{\pi_\theta} [(f(X_t, \alpha_t) + V^{\pi_\theta}(t+1, X_{t+1})) \nabla_\theta \log \pi_\theta(t, X_t, \alpha_t) | X_t = x], \end{aligned}$$

and thus

$$\begin{aligned} \mathbb{E}_{\pi_\theta} [\nabla_\theta V^{\pi_\theta}(t, X_t)] &= \mathbb{E}_{\pi_\theta} [\nabla_\theta V^{\pi_\theta}(t+1, X_{t+1})] \\ &\quad + \mathbb{E}_{\pi_\theta} [(f(X_t, \alpha_t) + V^{\pi_\theta}(t+1, X_{t+1})) \nabla_\theta \log \pi_\theta(t, X_t, \alpha_t)]. \end{aligned}$$

By iterating over t , and noting that $\nabla_\theta V^{\pi_\theta}(T, x) = 0$, $\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta V^{\pi_\theta}(0, X_0)]$, we deduce the representation (4). Finally, by the same baseline trick (3), we observe that

$$\begin{aligned} \mathbb{E}_{\pi_\theta} [V^{\pi_\theta}(t, X_t) \nabla_\theta \log \pi_\theta(t, X_t, \alpha_t)] &= \mathbb{E}_{\pi_\theta} [V^{\pi_\theta}(t, X_t) \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(t, X_t, \alpha_t) | X_t]] \\ &= \mathbb{E}_{\pi_\theta} [V^{\pi_\theta}(t, X_t) \nabla_\theta \int_A \pi_\theta(t, X_t, a) \nu(da)] = 0. \end{aligned}$$